LAPORAN PRAKTIKUM STRUKTUR DATA

“QUEUE”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadirat Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-4 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada pembahasan konsep struktur data Queue dengan implementasi LinkedList dan Array.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan penggunaan Queue dalam real Project. Melalui projek projek sederhana yang dilakukan pada praktikum ini membantu mengembangkan pemahaman akan Queue pada program.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Dzhillan Dzhalila

DAFTAR ISI

KATA PENGANTAR i
BAB I PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Tujuan 3
1.3 Manfaat 3
BAB II PEMBAHASAN 4
2.1 Dasar Teori 4
2.2 Program QueueArray_2511531001 4
2.2.1 Kode Program 4
2.3 Program QueueArrayDriver_2511531001 6
2.3.1 Kode Program 6
2.3.2 Output Program 8
2.4 Program ReverseData_2511531001 8
2.4.1 Kode Program 8
2.4.2 Output Program 9
2.5 Program IterasiQueue_2511531001 9
2.5.1 Kode Program 9
2.5.2 Output 10
2.6 Program QueueLinkedList_2511531001 10
2.6.1 Kode Program 10
2.6.2 Output 11
BAB III KESIMPULAN 12
DAFTAR PUSTAKA 13

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman bahasa Java, salah satu struktur data yang dapat digunakan adalah Queue. Queue sendiri sering digunakan untuk menyimpan data dimana memungkinkan data yang masuk pertama kali dapat dieksekusi pertama kali. Ini menjadikan urutan data yang dimasukkan ke dalam tumpukan stack akan menjadi sama ketika tumpukan tersebut dikeluarkan.

Dalam Komputer Science konsep stack sendiri digunakan dalam berbagai kebutuhan. Salah satu fungsinya digunakan untuk penjadwalan proses (Scheduling) atau buffer data ataupun algoritma pencarian. Jenis-jenis Queue, linear/simple queue, circular queue, priority queue dan double-Ended Queue (Deque). Struktur dara ini krusial untuk memastikan data diproses dalam urutan yang tepat.

1.2 Tujuan

1. Memahami konsep dasar dari struktur data Queue.
2. Mengidentifikasi penggunaan Queue dalam program.
3. Mengimplementasikan metode bawaan dari Queue.

1.3 Manfaat

1. Memberikan pengalaman langsung dalam mengimplementasikan Stack.
2. Melatih ketepatan dalam memilih struktur data untuk menghasilkan program yang lebih efisien dan terstruktur

BAB II
PEMBAHASAN

2.1 Dasar Teori

Queue adalah struktur data yang menyusun elemen elemen data dalam urutan linier . Prinsip dasar dari queue adalah First In First Out (FIFO) yang mana setiap data yang pertama kali dimasukkan ke dalam antrean akan menjadi yang pertama pula yang dikeluarkan. Pada struktur data ini, urutan pertama (data yang akan dikeluarkan) disebut Front atau Head. Sebaliknya, data pada urutan terakhir (data yang baru saja ditambahkan) disebut Back, Rear atau Tail. Proses untuk menambahkan data pada antrean disebut dengan Enqueue sedangkan proses untuk menghapus data dari antrean disebut dengan Dequeue.

Implementasi dari penerapannya yang paling cocok mengatur dan mengelola antrean tugas atau operasi secara efisien. Dalam sistem pemrograman, ia digunakan untuk menangani tugas-tugas seperti penjadwalan proses, antrean pesan dan manajemen sumber daya

2.2 Program QueueArray_2511531001

2.2.1 Kode Program

```
package pekan4_2511531001;

public class QueueArray_2511531001 {
	int front_1001, rear_1001, size_1001;
// deklarasi variabel front_1001(indeks pertama elemen yang
dimasukkan), rear_1001 (indeks elemen terakhir yang
dimasukkan), size_1001 (kapasitas array) dengan tipe data
integer.

	int capacity_1001;
// deklarasi variabel capacity_1001 dengan tipe data
integer

	int array_1001[];
// deklarasi variabel array_1001 dengan tipe data integer.

	public QueueArray_2511531001 (int capacity_1001) {
		this.capacity_1001 = capacity_1001;
		rear_1001 = capacity_1001 -1;
		array_1001 = new int [this.capacity_1001 ];
	}
```

```
// kelas ini dideklarasikan sebagai konstruktor dari
keempat variabel dan satu array.
// variabel rear_1001 diinisiasikan dengan nilai variabel
capacity_1001 - 1
//pendeklarasian array dengan tipe integer dengan kapasitas
yang sama dengan nilai capacity_1001


        boolean isFull_1001 (QueueArray_2511531001
queue_1001) {
                return (queue_1001.size_1001 ==
queue_1001.capacity_1001 );}
// method ini untuk mengembalikan nilai true jika ukuran
array sudah sama dengan kapasitas (capacity_1001)

        boolean isEmpty_1001 (QueueArray_2511531001
queue_1001) {
                return (queue_1001.size_1001 ==0);}
// method ini untuk mengembalikan nilai true jika ukuran
array sama dengan 0 atau array kosong

        void enqueue_1001 (int item_1001) {
                if(isFull_1001(this))
                        return;
                this.rear_1001 = (this.rear_1001 +1 ) %
this.capacity_1001 ;
                this.array_1001 [this.rear_1001] = item_1001;
                this.size_1001 =this.size_1001 +1;
                System.out.println(item_1001 + " equeued to
queue");
                }
// method ini menambahkan elemen bar uke dalam antrian
dengan memanggil method isFull terlebih dahulu, jika queuqe
sudah penuh, maka method akan return tanpa melakukan apapun
//operator modulo memastikan bahwa jika rear sudah berada
pada indeks terkahir, penambahan 1 akan mengembalikannya ke
indeks 0
//elemen item_1001 disimpan di posisi rear yang baru dan
size_1001 ditambah satu


        int dequeue_1001() {
                if (isEmpty_1001(this))
                        return Integer.MIN_VALUE;
                int item_1001 = this.array_1001
[this.front_1001 ];
                this.front_1001 = (this.front_1001 +1 ) %
this.capacity_1001 ;
                this.size_1001 = this.size_1001 -1;
                return item_1001;
        }
// method ini akan mengambil dan menghapus elemen paling
depan dari antrian.
//memanggil method isEmpty, jika array kosong, maka method
akan mengembalikan integer.MIN_value sebagai kode error.
//mengambil data nilai dari array_1001[front_1001] ke
variabel sementara
```

```
//menggunakan modulo untuk memindahkan penunjuk front ke
indeks berikutnya secara melingkar
//mengubah kapasitas dengan mengurangkan size_1001 dengan 1
lalu data dikembalikan dengan return

	int front_1001 () {
		if (isEmpty_1001(this))
			return Integer.MIN_VALUE ;
		return this.array_1001 [this.front_1001 ];
		}

//method ini akan mengembalkan nilai elemen di posisi
front_1001

	int rear_1001 () {
		if (isEmpty_1001(this))
			return Integer.MIN_VALUE ;
		return this.array_1001 [this.rear_1001 ];
		}
//method ini akan mengembalikan nilai elemen di posisi
rear_1001

	void display_1001 () {
		int i ;
		if (front_1001 == rear_1001) {
			System.out.printf("\nAntrian
Kosong\n");
			return;
		}
//method ini akan menampilkan isi antrian
//mengecek apakah antrian kosong dengan nilai true untuk
front_1001 == rear_1001, dan akan menampilkan antrian
kosong


		for ( i = front_1001; i < rear_1001; i++) {
		System.out.printf(" %d <-- ", array_1001[i]);
	}
		return;
	}
//pengulangan ini akan terus terjadi jika i yang diisi
dengan nilai ideks elemen paling depan dan jika i masih
lebih kecil dari nilai indeks elemen paling terakhir maka
isi antrian akan bisa ditampilkan.
	}
```

Kode Program 2.1 : Kode ADT QueueArray dan Komentar sebagai penjelasannya.

2.3 Program QueueArrayDriver_2511531001

2.3.1 Kode Program

```
package pekan4_2511531001;

public class QueueArrayDriver_2511531001 {
```

```
        public static void main (String [] args) {
                QueueArray_2511531001 queue_1001 = new
QueueArray_2511531001(1000);
//membuat objek antrian

                queue_1001.enqueue_1001(10);
                queue_1001.enqueue_1001(20);
                queue_1001.enqueue_1001(30);
                queue_1001.enqueue_1001(40);
//memanggil method enqueque untuk menambahkan elemen yang
ada di dalam kurung untuk dimasukkan ke antrian

                System.out.println("Item di depan " +
queue_1001.front_1001() );
// memanggil method front_1001 untuk mengembalikan nilai
elemen paling depan dari antrian

                System.out.println("Item paling belakang " +
queue_1001.rear_1001());
//memanggil method rear_1001 untuk mengembalikan nilai
elemen paling belakang dari antrian (elemen yang terakhir
dimasukkan)
                System.out.println("tampilan queue");
                queue_1001.display_1001();
//memanggil method display_1001 untuk menampilkan isi dari
antrian
                System.out.println();
//menampilkan baris kosong

                System.out.println(queue_1001.dequeue_1001()
+ " dihapus dari queue");
//memanggil method dequeque untuk menghapus elemen pertama
yang dimasukkan ke dalam antrian atau queue

                System.out.println("Item di depan : " +
queue_1001.front_1001() );
//memanggil method front_1001 untuk menampilkan elemen
paling depan dari antrian setelah adanya penghapusan

                System.out.println("Item paling belakang  : "
+ queue_1001.rear_1001());
//memanggil mmethod rear_1001 untuk menampilkan elemen
paling belakang dari antrian setelah adanya penghapusan

                System.out.println("tampilan queue setelah
satu data dihapus");
                queue_1001.display_1001();
// menampilkan isi antrian setelah adanya penghapusan
        }
}
```

Kode Program 2.2 : kode driver QueueArrayDriver

2.3.2 Output Program

```
<terminated> QueueArrayDriver_2511531001 [Java Application]
10 equeued to queue
20 equeued to queue
30 equeued to queue
40 equeued to queue
Item di depan 10
Item paling belakang 40
tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
Item di depan : 20
Item paling belakang  : 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

Gambar 2.1 : Output Program QueueArrayDriver

2.4 Program ReverseData_2511531001

2.4.1 Kode Program

```
package pekan4_2511531001;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
// memanggil library java.util untuk LinkedList, Queue dan
juga Stack

public class ReverseData_2511531001 {
       public static void main (String [] args) {
              Queue <Integer> q_1001 = new
LinkedList<Integer>();
//membuat objek antrian dengan implementasi LinkedList

              q_1001.add(1);
              q_1001.add(2);
              q_1001.add(3); // [1,2,3]
//menggunakan class add untuk menambahkan elemen ke dalam
antrian

              System.out.println("sebelum reverse = " +
q_1001);
//menampilkan antrian sebelum dibalikkan

              Stack <Integer> s_1001 = new Stack <Integer>
();
//membuat objek stack

              while (!q_1001.isEmpty()) { // Q -> S
                     s_1001.push(q_1001.remove());
                     }
//perulangan ini akan terus terjadi selama antrian q_1001
tidak kosong
```

```
//setelah itu stack akan terisi dari elemen dari antrian
yang sudah di remove

            while (!s_1001.isEmpty()) { // S -> Q
                  q_1001.add(s_1001.pop());
                  }
//perulangan ini akan terjadi selama stack s_1001 belum
kosong
//queue akan terisi dengan elemen teratas dari stack yang
sudah dipopkan

            System.out.println("sesudah reverse = " +
q_1001); // [3,2,1]
            }
//menampilkan isi antrian setelah dibalik menggunakan
langkah sebelumnya.
      }
```

Kode Program 2.3 : kode program ReverseData

2.4.2 Output Program

```
<terminated> ReverseData_2511531001 [Java Appli
sebelum reverse = [1, 2, 3]
sesudah reverse = [3, 2, 1]
```

Gambar 2.2 : output program ReverseData

2.5 Program IterasiQueue_2511531001

2.5.1 Kode Program

```
package pekan4_2511531001;
import java.util.;
//memanggil library java.util

public class IterasiQueue_2511531001 {
      public static void main (String [] args) {
            Queue<String> q_1001 = new LinkedList <>();
//membuat objek dari antrian yaitu q_1001

            q_1001.add("Praktikum");
            q_1001.add("Struktur");
            q_1001.add("Data");
            q_1001.add("Dan");
            q_1001.add("Algoritma");
//menggunakan class add untuk menambahkan elemen ke
antrian

            Iterator<String> iterator_1001 =
q_1001.iterator();
```

```
//method iterator dipanggil pada objek q_1001 untuk
menghasilkan objek iterator yang siap menelurusi koleksi
tersebut

            while (iterator_1001.hasNext()) {
                  System.out.print(iterator_1001.next() +
" ");
//perulangan ini akan terus terjadi selama iterator_1001
yang bergantung kepada hasNext bernilai true jika masih
ada elemen berikutnya yang belum diakses dan false jika
iterator sudah mencapai akhir koleksi
//perulangan ini akan berhenti jika iterator sudah
mencapai akhir dari solusi
            }
      }
}
```

Kode Program 2.4 : Kode Program IterasiQueue

2.5.2 Output

```
Javadoc  Declaration  Console ×  Progress
<terminated> IterasiQueue_2511531001 [Java Application] C:\Users\ASUS\.p
Praktikum Struktur Data Dan Algoritma
```

Gambar 2.3 : output program IterasiQueue

2.6 Program QueueLinkedList_2511531001

2.6.1 Kode Program

```
package pekan4_2511531001;
import java.util.;
//memanggil library java.util

public class QueueLinkedList_2511531001 {
      public static void main (String [] args) {
            Queue <Integer> q_1001 = new LinkedList <>
();
//membuat objek antrian q_1001 menggunakan LinkedList

//tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
            for (int i = 0; i < 6; i++) {
                  q_1001.add((i));
            }
//Menampilkan isi antrian
            System.out.println("Elemen Antrian " +
q_1001);

//untuk menghapus kepala antrian.
            Int hapus_1001 = q_1001.remove();
```

```
//untuk menampilkan elemen yang dihapus
            System.out.println("Hapus Elemen = " +
hapus_1001);

//untuk menampilkan antrian yang sudah dihapus
            System.out.println(q_1001);

//untuk melihat antrian terdepan
            int depan_1001 = q_1001.peek();
            System.out.println("Kepala Antrian = " +
depan_1001);

//untuk menampilkan size dari antrian
            int banyak_1001 = q_1001.size();
            System.out.println("Size Antrian = " +
banyak_1001);
            }
      }
```

Kode Program 2.5 : Kode Program QueueLinkedList

2.6.2 Output

```
<terminated> QueueLinkedList_2511531001 [Java Applic
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus Elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2.4 : output program QueueLinkedList

BAB III
KESIMPULAN

Berdasarkan kelima program yang telah dibahas, konsep fundamental struktur data Queue dapat disimpulkan sebagai mekanisme pengelolaan data yang menganut prinsip First-In-First-Out (FIFO), di mana elemen yang pertama kali dimasukkan akan selalu menjadi elemen pertama yang diproses atau dikeluarkan, menjamin konsistensi urutan pemrosesan data. Konsep ini termanifestasi melalui operasi dasar enqueue (penambahan di rear) dan dequeue (penghapusan dari front) yang dapat diimplementasikan secara fleksibel, baik menggunakan array dengan teknik circular buffering untuk efisiensi memori statis maupun menggunakan LinkedList untuk kapasitas dinamis. Selain itu, Queue menunjukkan sifat interoperabilitas yang kuat ketika dikombinasikan dengan struktur data lain seperti Stack untuk manipulasi data kompleks (misalnya pembalikan urutan) serta mendukung teknik penelusuran aman menggunakan Iterator tanpa mengganggu integritas antrian, menegaskan bahwa Queue bukan sekadar wadah penyimpanan, melainkan alat kontrol alur data yang esensial untuk menciptakan sistem pemrosesan yang terstruktur, adil, dan efisien dalam berbagai skenario pemrograman.

DAFTAR PUSTAKA

[1] P. Prasatya, "Apa itu stack dalam ilmu pemrograman," *CodePolitan*, 2025. [Online]. Tersedia: https://www.codepolitan.com/blog/apa-itu-stack-dalam-ilmu-pemrograman/ . [Diakses: 29-Apr-2026].

[2] R. Maulana, "Struktur data queue: Pengertian, fungsi, dan jenisnya," *Dicoding*, 2023. [Online]. Tersedia: https://www.dicoding.com/blog/struktur-data-queue-pengertian-fungsi-dan-jenisnya/ . [Diakses: 29-Apr-2026].

[3] "Struktur data queue: Pengertian, jenis & contoh struktur data queue dalam mengolah data," *Course-Net*, 2024. [Online]. Tersedia: https://course-net.com/blog/struktur-data-queue-pengertian-jenis-contoh-struktur-data-queue-dalam-mengolah-data/ . [Diakses: 29-Apr-2026].

LAPORAN TUGAS

1. AntrianLoket_2511531001

   a. Program

```java
package pekan4_2511531001;

public class AntrianLoket_2511531001 {
    // Variabel sesuai ketentuan tugas + suffix _1001
    String[] queue_1001;
    int front_1001;
    int rear_1001;
    int max_1001;
    int count_1001;
// Variabel bantu untuk melacak jumlah elemen aktif

    // Konstruktor inisialisasi array & indeks
    public AntrianLoket_2511531001(int max_1001) {
            this.max_1001 = max_1001;
            this.queue_1001 = new String[this.max_1001];
            this.front_1001 = 0;
            this.rear_1001 = -1;
            this.count_1001 = 0;
    }

    // 1. isEmpty(): Cek apakah antrian kosong
    public boolean isEmpty_1001() {
            return count_1001 == 0;
    }

    // 2. isFull(): Cek apakah antrian penuh
    public boolean isFull_1001() {
            return count_1001 == max_1001;
    }

    // 3. enqueue(): Tambah data ke belakang antrian
    public void enqueue_1001(String nama_1001) {
            if (isFull_1001()) {
                    System.out.println("Antrian sudah penuh!");
                    return;
            }
            // Circular queue logic
            rear_1001 = (rear_1001 + 1) % max_1001;
            queue_1001[rear_1001] = nama_1001;
            count_1001++;
            System.out.println("Data berhasil ditambahkan ke
antrian");
    }

    // 4. dequeue(): Hapus data dari depan antrian
    public void dequeue_1001() {
            if (isEmpty_1001()) {
                    System.out.println("Antrian kosong!");
                    return;
            }
```

```
            String nama_1001 = queue_1001[front_1001];
            front_1001 = (front_1001 + 1) % max_1001;
            count_1001--;
            System.out.println(nama_1001 + " telah dilayani");
    }

    // 5. display(): Tampilkan seluruh isi antrian
    public void display_1001() {
            if (isEmpty_1001()) {
                    System.out.println("Isi antrian: Kosong");
                    return;
            }
            System.out.println("Isi antrian:");
            for (int i_1001 = 0; i_1001 < count_1001; i_1001++)
{
                    int idx_1001 = (front_1001 + i_1001) %
max_1001;
                    System.out.println((i_1001 + 1) + ". " +
queue_1001[idx_1001]);
            }
    }

    // 6. reverse(): Balik urutan seluruh isi antrian
    public void reverse_1001() {
            if (isEmpty_1001()) {
                    System.out.println("Antrian kosong, tidak
dapat dibalik.");
                    return;
            }
            String[] temp_1001 = new String[max_1001];
            int idx_1001 = front_1001;

            // Salin elemen ke array sementara
            for (int i_1001 = 0; i_1001 < count_1001; i_1001++)
{
                    temp_1001[i_1001] = queue_1001[idx_1001];
                    idx_1001 = (idx_1001 + 1) % max_1001;
            }

            // Balik urutan di array sementara
            for (int i_1001 = 0; i_1001 < count_1001 / 2;
i_1001++) {
                    String swap_1001 = temp_1001[i_1001];
                    temp_1001[i_1001] = temp_1001[count_1001 - 1
- i_1001];
                    temp_1001[count_1001 - 1 - i_1001] =
swap_1001;
            }

            // Kembalikan ke queue utama & reset indeks ke
bentuk linier
            front_1001 = 0;
            rear_1001 = count_1001 - 1;
            for (int i_1001 = 0; i_1001 < count_1001; i_1001++)
{
                    queue_1001[i_1001] = temp_1001[i_1001];
            }}}
```

2. AntrianLoketDriver_2511531001

   a. Program

```
package pekan4_2511531001;

import java.util.Scanner;

public class AntrianLoketDriver_2511531001 {
    public static void main(String[] args) {
            Scanner sc_1001 = new Scanner(System.in);
            // Inisialisasi objek antrian dengan kapasitas
default 100
            AntrianLoket_2511531001 antrian_1001 = new
AntrianLoket_2511531001(100);
            int pilihan_1001 = 0;

            do {
                    System.out.println("\n=== PROGRAM ANTRIAN
LOKET");
                    System.out.println("1. Tambah Antrian");
                    System.out.println("2. Hapus Antrian");
                    System.out.println("3. Tampilkan Antrian");
                    System.out.println("4. Reverse");
                    System.out.println("5. Keluar");
                    System.out.print("Pilih menu: ");

                    // Validasi input menu
                    if (sc_1001.hasNextInt()) {
                            pilihan_1001 = sc_1001.nextInt();
                            sc_1001.nextLine(); // Bersihkan
karakter newline
                    } else {
                            System.out.println("Input tidak valid.
Silakan masukkan angka.");
                            sc_1001.next();
                            continue;
                    }

                    // Routing menu ke method yang sesuai
                    switch (pilihan_1001) {
                            case 1:
                                    System.out.print("Masukkan nama
pelanggan: ");
                                    String nama_1001 =
sc_1001.nextLine();

    antrian_1001.enqueue_1001(nama_1001);
                                    break;
                            case 2:
                                    antrian_1001.dequeue_1001();
                                    break;
                            case 3:
                                    antrian_1001.display_1001();
                                    break;
                            case 4:
                                    antrian_1001.reverse_1001();
```

```
                                antrian_1001.display_1001(); //
Tampilkan hasil reverse sesuai contoh output
                                break;
                        case 5:
                                System.out.println("Program
selesai");
                                break;
                        default:
                                System.out.println("Pilihan
tidak tersedia.");
                }
        } while (pilihan_1001 != 5);

        sc_1001.close();
    }
}
```

b. Ouput

```
=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Zaza
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Lathifa
Data berhasil ditambahkan ke antrian
```

```
=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Zaza
2. Lathifa

=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Zaza telah dilayani
```

```
=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Lathifa

=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Aliifah
Data berhasil ditambahkan ke antrian
```

```
=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Lathifa
2. Aliifah

=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Isi antrian:
1. Aliifah
2. Lathifa

=== PROGRAM ANTRIAN LOKET
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```